Text-to-SQL con LoRA

Proyecto 3 — Deep Learning 2026

Cindy Gualim · (segundo integrante)
Universidad del Valle · Qwen2.5-Coder-1.5B-Instruct + LoRA

El problema

Dado el esquema de una base (SHOW CREATE TABLE, con sus COMMENT) y una pregunta en espanol, devolver una consulta MySQL 8.4.10 correcta.

Se mide por exactitud por ejecucion: se comparan los resultados, no el texto.

El docente evalua sobre bases y preguntas que no conocemos.

Las condiciones que mandan en el diseno

- ONLYFULLGROUP_BY activo: toda columna no agregada va en el GROUP BY.
- 30 s por pregunta, 5 min para cargar el modelo.
- Generacion determinista.
- Una consulta que modifique la base cuesta 10 puntos.
- La particion entre entrenamiento y validacion se hace por esquema.

Cada una de estas condiciones aparece como una decision concreta en el codigo.

El dataset: 12 esquemas propios

6 a 7 tablas cada uno, con COMMENT en tabla y columna.

Mezclados a proposito, porque el enunciado anuncia esas condiciones:

- 5 en espanol, 3 en ingles, 2 abreviados (pac, med, cita; mbr, cls, bkg)
- clave primaria no numerica (aeropuertos.iata), tabla puente, autorreferencias

Datos generados por un poblador generico que lee information_schema,
ordena las tablas por dependencias y usa semilla fija: 9,525 filas reproducibles.

32 plantillas en tres niveles

Nivel	Plantillas	Construcciones
facil	11	una tabla, filtro, agregado, DISTINCT,
media	10	JOIN, GROUP BY, HAVING, doble JOIN
dificil	11	subconsulta correlacionada, WITH, ROW_

Los umbrales de los filtros salen de los datos: p25, p50 y p75 de una muestra real de cada tabla. Un filtro que no coincide con ningun dato produce un par inutil.

Validacion por ejecucion: 2,464 pares

Se ejecutaron todos con un usuario de solo lectura, como el del evaluador.

```
generados    2,464
validos      2,391   97.0%
descartados     73   todos por devolver cero filas
```

Cero errores de sintaxis. Cero violaciones de ONLYFULLGROUP_BY.

Un resultado vacio se descarta a proposito: con cero filas, cualquier consulta rota que tambien devuelva vacio "acierta".

El hallazgo que cambio el dataset

La deteccion de preguntas duplicadas no encontro repeticion, encontro ambiguedad real:

- vuelos tiene dos claves foraneas a aeropuertos: origen y destino.
- prestamos apunta a clientes y a sucursales, y ambas tienen nombre.

Lo grave no era el par descartado sino el que se quedaba: entrenaba al modelo a adivinar.

Se agrego desambiguacion automatica. Preguntas ambiguas: 19 $\rightarrow$ 13 $\rightarrow$ 0.

Validacion propia

2 esquemas nuevos, veterinaria (espanol) y hotel (ingles), **sin ninguna interseccion** con el entrenamiento, ni siquiera un nombre de tabla.

40 preguntas escritas a mano: 13 faciles, 13 medias, 14 dificiles. Peso 81.

Escribirlas a mano encontro dos defectos que ninguna prueba automatica habria encontrado: perros con fecha de nacimiento de persona, y score de resena llegando a 100 cuando su COMMENT dice 1 a 5.

Dos configuraciones de LoRA

Difieren en un solo factor: r = 8 contra r = 32.

alpha = 32 fijo en ambas. Escalarlo con el rank cambiaria a la vez la capacidad y la magnitud de la contribucion, y no se podria atribuir la diferencia.

Hipotesis previa: A alcanza para el formato y el anclaje al esquema, que es el error dominante del modelo base; B solo deberia ganar en el nivel dificil, con riesgo de sobreajuste a las 32 plantillas.

(resultados: parametros entrenables, tiempo y curvas)

Experimento de inferencia

Tres variantes sobre el mismo adapter:

1. directa — el prompt del contrato
2. few-shot — tres ejemplos resueltos sobre un esquema inventado
3. autocorreccion — ejecuta la consulta y la reescribe con el error del motor

La autocorreccion solo puede arreglar lo que el motor reporta:
Unknown column, Table doesn't exist. No hace nada por resultado_distinto,
donde la consulta corre y nada indica que este mal. Ese es su limite duro.

Analisis de errores

Se reporta el numero de preguntas falladas y el peso perdido, que no es lo mismo: fallar una dificil cuesta lo mismo que fallar tres faciles.

Categorias: sintaxis · tabla inexistente · columna inexistente · join · agregacion · filtro · resultado distinto.

Las dos primeras familias salen del codigo de error de MySQL; las demas, de comparar la forma de la consulta generada contra la de referencia.

Prediccion de la EX oculta

0.52, en un rango de 0.45 a 0.60.
Mejora sobre el modelo sin adapter: +0.20 a +0.30.

Descontamos 0.15 de nuestra validacion propia: la escribimos nosotros, sobre esquemas que diseniamos nosotros.

La limitacion mas seria que asumimos: el dataset viene de plantillas, no de lenguaje natural libre. La diferencia entre las dos versiones es la parte de la prediccion de la que estamos mas seguros, porque se mide sobre las mismas preguntas.